Task Brief: Local Screenshot Persistence & Data Routing

Objective
Implement local file persistence for blurred screen captures and route metadata into SQLite tables while keeping binary image data off the database file.

Task Requirements & Acceptance Criteria

Work Area	Technical Requirement	Acceptance Criteria
Disk Storage	• Create a dedicated `screenshots/` directory inside the app's local data folder. • Generate a unique UUID v4 identifier for each capture. • Export the in-memory blurred image buffer directly to disk as `{uuid}.jpg`.	• Images saved cleanly to disk as JPEGs. • Zero raw binary image data stored in the SQLite file.
Integrity Check	• Compute a SHA-256 hash fingerprint of the blurred image buffer in memory before writing to disk.	• SHA-256 hash generated for data verification.

Database Routing	• Execute an atomic transaction to write two records simultaneously: 1. Log event details (`app_name`, `sanitized_title`, `duration`, `is_idle`) into `activity_queue`. 2. Log file pointers (`screenshot_id`, `local_file_path`, `image_hash`, timestamp) into `screenshot_logs`.	• Both records commit together. • Transaction rolls back completely if file write or insert fails.

Checklist

- [ ] Verify the blurred image frame remains strictly in memory (RAM) until saved to disk.
- [ ] Ensure the file system directory creates automatically on app launch if missing.
- [ ] Confirm that `screenshot_logs.local_file_path` contains valid path references to disk files.
- [ ] Verify that saving 50+ screenshots does not cause the SQLite database file size to balloon.
- [ ] Test atomic rollback by triggering a write error and verifying no orphan entries land in SQLite.

For Detailed Explaination Refer the PRD Document.